

Fish Tracking

Laboratory of Computational Physics Mod. B
Supervisor: Professor Carlo Albert

Contents

1	Introduction to the Problem	3
2	Our approach more in detail	4
2.0.1	Toy bathymetry toy data	4
2.0.2	Real bathymetry toy data	5
2.0.3	Real bathymetry real data	5
3	Sampling	9
3.1	Parallel Tempering (PT)	9
3.1.1	The Setup	9
3.1.2	The Algorithm	10
3.1.3	Performance Estimator	10
3.1.4	Global Communication Barrier in Parallel Tempering	10
3.2	MCMCTempering.jl	11
3.3	Pigeons.jl	11
4	Description of Pigeons' Samplers	12
4.1	AutoMALA Sampler	13
4.1.1	Connection between AutoMALA and HMC samplers	13
4.1.2	Mathematical Background	14
4.1.3	How AutoMALA Works	14
4.2	Slice Sampler	15
4.2.1	Overview and Motivation	15
4.2.2	Single-Variable Slice Sampling	16
4.2.3	Multivariate Slice Sampling	16
5	Explorers comparison	17
5.1	AutoMALA in Pigeons	17
5.1.1	Grid Search for Parameter Tuning	17
5.2	Slice Sampler in Pigeons	18
5.2.1	Grid Search for Parameter Tuning.	19
5.3	AutoMALA vs Slice Sampler	19

6	Sampling results	21
6.1	Toy bathymetry toy data	21
6.2	Real Bathymetry & Toy Data	25
6.3	Real Bathymetry & Real Data	26
6.3.1	Persisting Issues	29

1 Introduction to the Problem

Understanding the movements and migrations of marine animals, particularly fish, is a fundamental challenge in spatial ecology. This knowledge is essential for explaining observed ecological patterns, predicting responses to environmental changes, and informing effective management strategies for fisheries and conservation. Direct observation of individual fish movements over large scales is rarely feasible due to the vast and opaque nature of aquatic environments. Instead, researchers rely on indirect measurements collected via electronic data storage tags attached to the animals. These tags record environmental variables that vary spatially, such as light intensity, water pressure (depth), temperature, salinity, or acoustic signals, which can be used to infer positions post-recovery or transmission.

Measurements from these tags are inherently noisy and indirect, stemming from sensor inaccuracies, environmental variability, or incomplete spatial coverage. For instance, pressure sensors provide depth readings that must be matched to known bathymetry maps, but uncertainties arise from tidal fluctuations, sensor drift, or imprecise bathymetric data. Similarly, acoustic tags detect proximity to fixed receivers, but detection probabilities decrease with distance due to signal attenuation in water. To address these uncertainties, a statistical framework is necessary to reconstruct plausible trajectories, providing not only point estimates but also measures of uncertainty.

A typical statistical approach combines a movement model, which describes the animal's likely displacements, with a measurement model, which links observations to positions. The movement model often assumes a (un)biased random walk. The measurement model specifies the likelihood of observed data given a position, accounting for error distributions. The goal is to infer the posterior distribution of the trajectory given all observations, typically within a state-space modeling framework.

One established method for this inference, as detailed in Thygesen et al. (2009) [1], employs Hidden Markov Models (HMMs) on a discretized spatial grid. However, HMMs can suffer from computational intensity on fine grids, limited exploration of multimodal posteriors (e.g., when multiple routes are plausible), and challenges in sampling full trajectories for complex queries.

In our project, we address the problem of inferring an unknown fish trajectory using a Bayesian sampling approach, specifically Parallel Tempering (PT) Markov Chain Monte Carlo (MCMC), to draw samples from the posterior distribution of positions. This approach mitigates the paper's HMM limitations by enabling direct trajectory sampling without grid discretization, better handling of high-dimensional spaces, and quantifying full-trajectory uncertainties (e.g., probability of entering regions). It is computationally efficient for our purposes and extensible to more complex models.

The core objective is to estimate the trajectory $\mathbf{X} = (X_1, X_2, \dots, X_T)$, where each $X_t = (x_t, y_t)$ is the 2D position at time t , given observations $\mathbf{Y} = (\mathbf{Y}_a, \mathbf{Y}_d)$. Here, $\mathbf{Y}_a$ are acoustic detections/non-detections from fixed receivers, and $\mathbf{Y}_d$ are depth measurements. The posterior is:

$$\log p(\mathbf{X} \mid \mathbf{Y}) = \log p(\mathbf{Y}_a \mid \mathbf{X}) + \log p(\mathbf{Y}_d \mid \mathbf{X}) + \log p(\mathbf{X}), \quad (1)$$

We sample from this posterior using PT.

The movement prior $p(\mathbf{X})$ models a random walk with independent Gaussian increments:

$$\log p(\mathbf{X}) = \sum_{t=2}^T [\log \mathcal{N}(x_t \mid x_{t-1}, \sigma) + \log \mathcal{N}(y_t \mid y_{t-1}, \sigma)], \quad (2)$$

where σ controls step size, ensuring plausible displacements.

For acoustic likelihood $p(\mathbf{Y}_a | \mathbf{X})$, each observation is a tuple (t, s, r) , where $s \in \{:\text{detect}, :\text{nondetect}\}$ and r is a receiver with position (r_x, r_y) , detection range d_0 , and sharpness k . The probability is sigmoid-based on distance to the receiver:

$$p(:\text{detect} | X_t, r) = 1 - \frac{1}{1 + \exp(-(d - d_0)/k)}, \quad p(:\text{nondetect} | X_t, r) = 1 - p(:\text{detect}), \quad (3)$$

where $d = \sqrt{(x_t - r_x)^2 + (y_t - r_y)^2}$. Thus,

$$\log p(\mathbf{Y}_a | \mathbf{X}) = \sum_{(t,s,r) \in \mathbf{Y}_a} \log p(s | X_t, r). \quad (4)$$

For depth likelihood $p(\mathbf{Y}_d | \mathbf{X})$, each observation is a tuple (t, z) , where z is the observed depth matched to the bathymetry-interpolated depth at X_t . The likelihood accounts for bathymetry errors using a normal distribution conditioned on the bathymetric depth to enforce realism:

$$p(z | X_t) = \mathcal{N}(z | z_{max}, \sigma), \quad (5)$$

where z_{max} is the bathymetric depth at the fish position X_t , and σ is the measurement noise. Thus,

$$\log p(\mathbf{Y}_d | \mathbf{X}) = \sum_{(t,z) \in \mathbf{Y}_d} \log(\mathcal{N}(z_{max}, \sigma), z). \quad (6)$$

2 Our approach more in detail

We tackled the problem of track-reconstruction using three successive approaches:

- toy bathymetry toy data (TT);
- real bathymetry toy data (RT);
- real bathymetry real data (RR);

In each of them the idea was to use an appropriate initial trajectory, which would be optimized through appropriate sampling in the logposterior space.

2.0.1 Toy bathymetry toy data

The purpose of this configuration is to work in a fully controllable scenario. It starts from hand-drawn channel-like bathymetries.

To get a proper initial trajectory we focused on the fact that from the acoustic data we know, approximately, two regions through which the fish passed on its trajectory, which we will call A and B. We then generate biased-random walks that start from A and arrive at B, in a fixed number of steps, to be consistent with the fact that the time of passage in B is known (but implicitly assuming the fish swims at constant speed). Any of these walks is then chosen as the initial trajectory (Figure 1).

To get the data to work with we pick another of the walks and we consider that on as the real path the fish followed. We extract the bathymetry values associated to it and once perturbed with uniform noise (to simulate measurements errors) we treat them as true depth data. From the same trajectory we generate acoustic data too: as long as the trajectory is within the activation zone

of a receiver, that receiver stores positive values, while, for trajecotry steps outside that area, the receiver stores 0 (Figure 1).

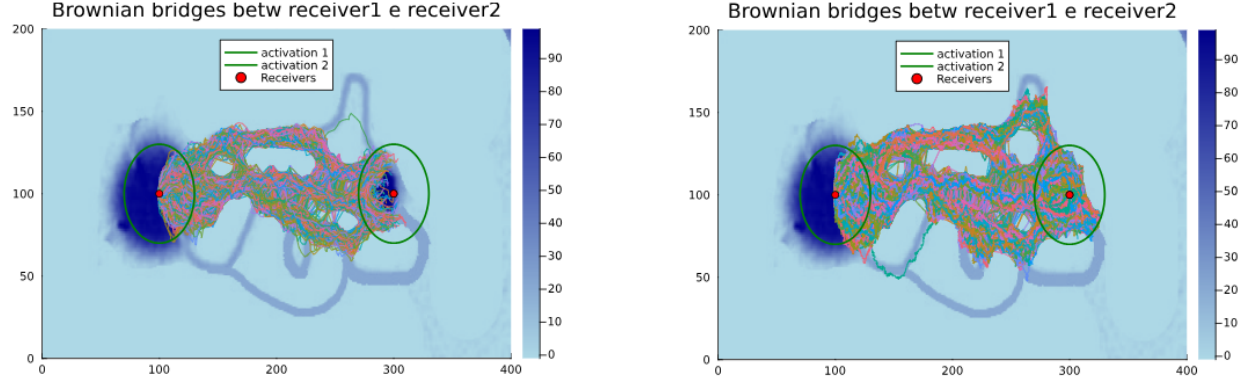


Figure 1: Biased random walks from receiver 1 to receiver 2 made respectively of 100 and 400 steps (from left to right)

2.0.2 Real bathymetry toy data

To move towards a more real scenario we used the same approach to get the initial trajecotry and to generate data, but using a portion of the real bathymetry Figure 2.

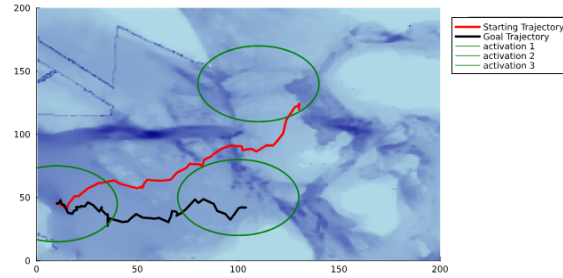


Figure 2: A real bathymetry scenario

2.0.3 Real bathymetry real data

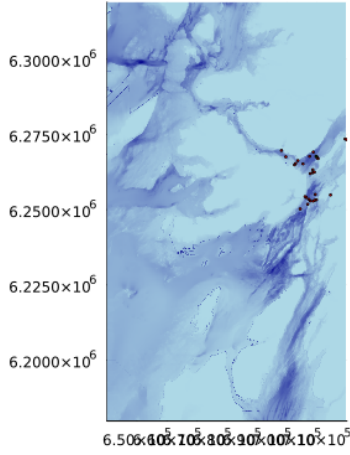
Once tested the algorith (that will be presented below) in the other two scenarios, this is the real life application. Here, however, the approach to be followed is different, since both the depth and acoustic data are already available.

The trajectory preprocessing workflow leverages several specialized functions to prepare acoustic and depth observation data for subsequent analysis with the Pigeons parallel tempering algorithm. The process is as follows:

1. Acoustic Data Import:

The function `load_acoustic_data` is responsible for importing and formatting both the positions of the acoustic receivers and the associated detection observations. Receiver positions are extracted and stored in both tuple and NamedTuple formats to facilitate downstream processing.

Positions of receivers on the total bathymetry



Receivers in the Zoomed Bathymetry

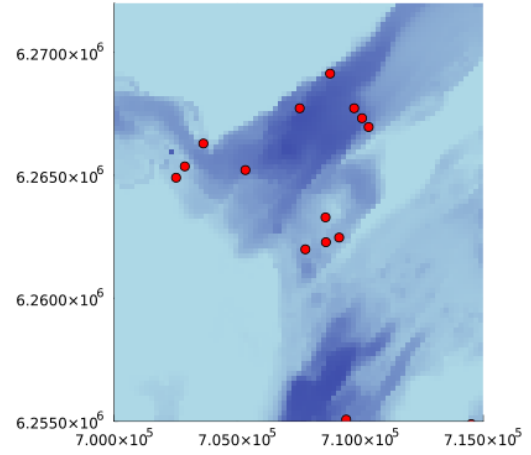


Figure 3: In these plots we can see the positions of the receivers in different bathymetries. On the left on a wide shot one. On the right in a zoomed-in map that concerns and will be used from now ahead

2. Receiver Activation Sequence Construction:

The function *'build_receiver_sequence'* processes the acoustic data to identify the activation events of each receiver throughout the observation period. It returns the chronological sequence of activated receivers, the corresponding time steps for each activation, and the indices of the events. (Figure 4)

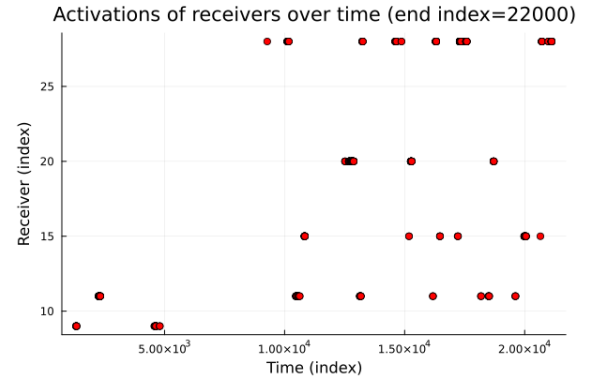
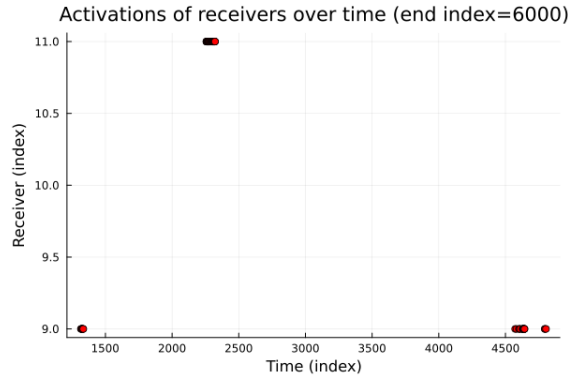


Figure 4: Plot of the different activations of the receivers, designed by their indexes over time. On the left side is during the first 6000 timesteps and on the right side is during all the dataset

3. Trajectory Refinement via Missing Data Interpolation:

In our case, the starting trajectory is obtained from the Starfish [6] package, provided by Professor Andreas Scheidegger, and just makes use of the receivers activation sequence, their position, the depth signals and the bathymetry itself; the problem with it, is that it may include points with missing ('NaN') coordinates (Table 1). The function *'fill_nan_runs_with_spline'* detects contiguous runs of missing data within the trajectory and performs linear interpolation between the nearest valid points before and after each run. This step ensures the continuity and completeness of the trajectory for further processing.(Figure 5)

InitialIndex	FinalIndex	Length
1	1315	1315
2258	2316	59
2319	4572	2254
10468	10826	359
13135	13233	99
16178	16290	113
16469	17289	821
18180	18697	518
19610	20701	1092
21124	23040	1917

Table 1: Different NaN runs, with their respective Initial Index, Final Index and length

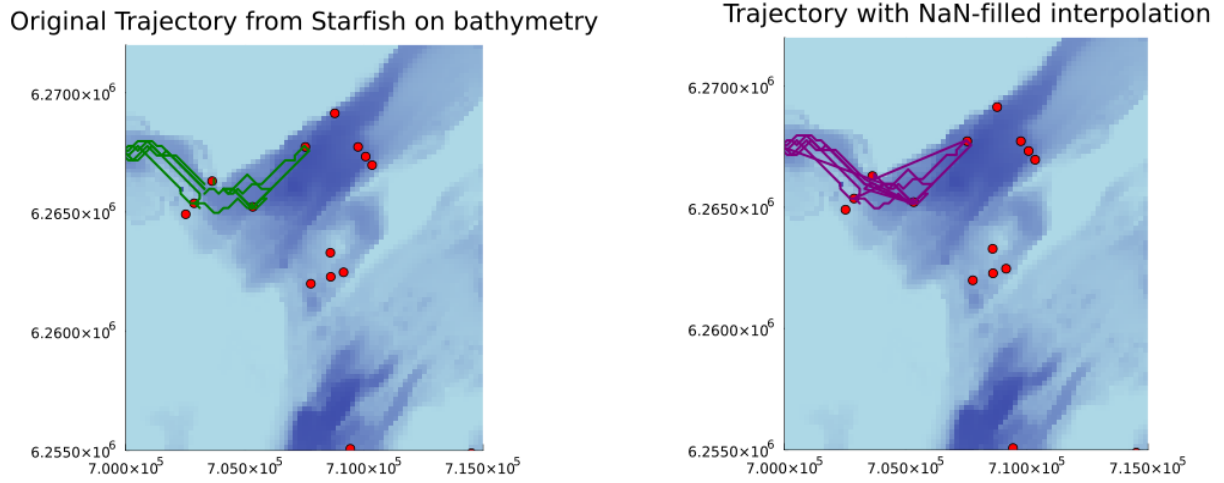


Figure 5: Plot of the initial trajectories in the first stages. On the left side we can see the Original initial Trajectory created with the starfish package. On the right side, the trajectory is shown after the first Refinement and processing, this means a linear interpolation between non-NaN points.

4. Elimination of Consecutive Duplicate Points:

In fact, for further processing, we will stop working with the whole dataset trajectory, as is computationally more expensive and it would be just a matter of applying the successive workflow to the desired portion of data. Instead we will just select an intermediate interval of the dataset of around 6000 time steps the dataset, where different receivers are activated and continue working with their corresponding trajectory. To mitigate issues that arise from repeated points in the trajectory (which are problematic for the Pigeons algorithm), the function `'remove_consecutive_duplicates'` compresses consecutive duplicate coordinates into single representative points (Figure 6) This yields a set of fundamental trajectory points that preserve the essential structure of the original path; in our case the almost 6000 points turned out to be "only" 26 fundamental points.

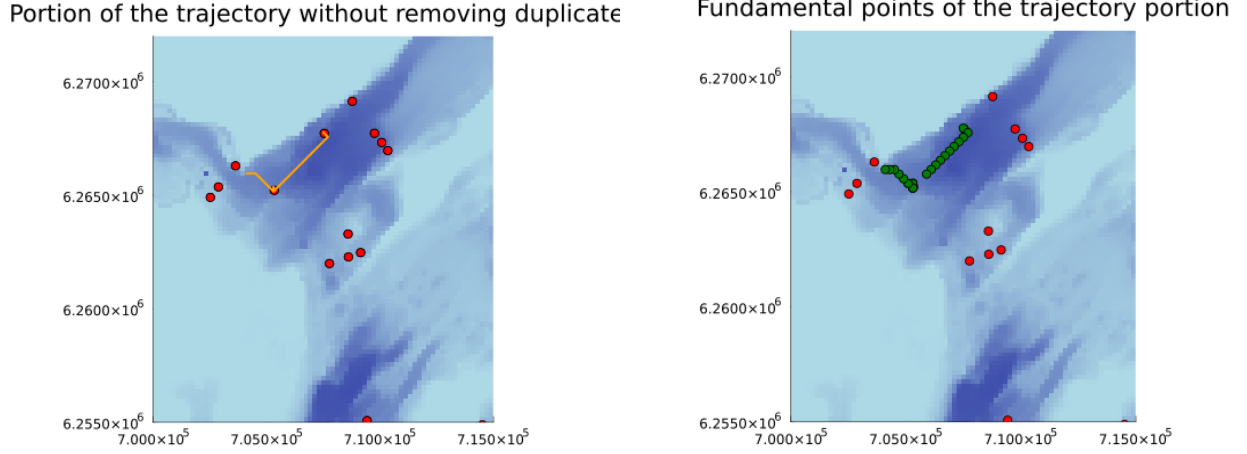


Figure 6: On the left: Plot of the raw portion of the trajectory (around 6000 time steps); On the right: Plot of the points that compose that same trajectory (26 fundamental points)

5. Trajectory Expansion and Randomization:

The deduplicated trajectory is typically shorter than the desired temporal resolution. The function *'make_noisy_linear_trajectory'* linearly interpolates between the fundamental points, allocating the appropriate number of interpolated points per segment to match the required trajectory length. Gaussian noise is added to each point to avoid artificial regularity. (Figure 7) Additionally, a bathymetric filter ensures that all generated points are physically valid (i.e., located in water).

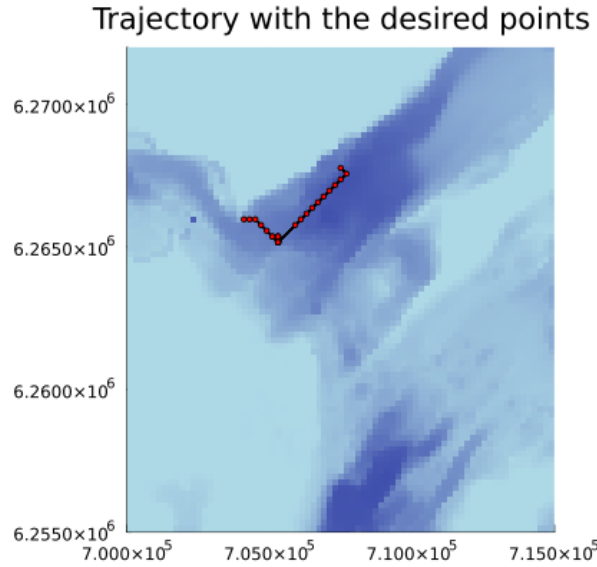


Figure 7: Portion of the random initial trajectory fully processed and crafted from the fundamental points

6. Integration in Analysis Pipeline:

These preprocessing steps are integrated in sequence within the main analysis pipeline. The

result is a dimensionally consistent, continuous, and physically valid trajectory that is compatible with the requirements of the Pigeons parallel tempering algorithm for fish tracking inference.

Now that we have a proper initial trajectory, we're left with the sampling procedure.

3 Sampling

As we already saw, the problem of track reconstruction can be cast in a Bayesian inference framework. The inference problem consists infact of estimating the posterior distribution of trajectories conditional on the observed depth and acoustic data:

$$p(S \mid Y_{\text{depth}}, Y_{\text{acoustic}}, \text{bathymetry}) \propto p(Y_{\text{depth}} \mid S, \text{bathymetry}) p(Y_{\text{acoustic}} \mid S) p(S), \quad (7)$$

where S denotes a candidate trajectory and $p(S)$ encodes the movement prior.

The likelihood, however, is multimodal due to repeating bathymetric features, and the imposed constraints further complicate the geometry of the feasible state. As a result, this posterior turned out to be computationally intractable to explore using naive enumeration or standard Monte Carlo, or even Simulated Annealing. The trajectory reconstruction problem therefore required the use of a more advanced sampling method capable of efficiently exploring this posterior distribution, which turned out to be **Parallel Tempering**.

3.1 Parallel Tempering (PT)

Parallel tempering (PT) is a Markov chain Monte Carlo (MCMC) methodology designed to improve mixing when sampling from complex, multimodal, or otherwise intractable target distributions. Standard MCMC methods often become trapped in local regions of high probability mass, particularly when the target distribution has well-separated modes [3]. PT addresses this issue by introducing a family of intermediate distributions that gradually interpolate between a tractable reference distribution and the desired target distribution.

3.1.1 The Setup

Let π_1 denote the intractable target distribution of interest, and let π_0 be a reference distribution from which efficient independent sampling is possible. PT constructs a path of intermediate distributions $\{\pi_t : t \in [0, 1]\}$ that interpolates between π_0 and π_1 .

The traditional PT path is the following:

$$\pi_\beta(x) \propto \pi_0(x)^{1-\beta} \pi_1(x)^\beta, \quad \beta \in [0, 1]. \quad (8)$$

This is called the *power posterior path*, and it is linear in the log-densities of π_0 and π_1 .

In the Bayesian setting, the target distribution is the posterior, so we can choose the reference distribution as the prior, from which we can often obtain independent samples.

To implement PT in practice, the continuum of $\beta \in [0, 1]$ is discretized into a schedule:

$$T_N = (\beta_0, \beta_1, \dots, \beta_N), \quad 0 = \beta_0 < \beta_1 < \dots < \beta_N = 1, \quad (9)$$

with mesh size

$$\|T_N\| = \max_{n \in \{0, \dots, N-1\}} (\beta_{n+1} - \beta_n). \quad (10)$$

3.1.2 The Algorithm

Parallel tempering operates on $N + 1$ coupled Markov chains, each evolving according to one of the intermediate distributions π_{β_n} . The algorithm alternates between two types of updates:

- **Local Exploration:** each chain is updated independently via MCMC sampling (this step can be executed in parallel across chains).
- **Communication via swaps:** to propagate information between chains, adjacent states X_n and X_{n+1} are proposed to be swapped. A swap $(x_n, x_{n+1}) \mapsto (x_{n+1}, x_n)$ is accepted with probability:

$$\alpha_n = 1 \wedge \frac{\pi_{\beta_n}(x_{n+1}) \pi_{\beta_{n+1}}(x_n)}{\pi_{\beta_n}(x_n) \pi_{\beta_{n+1}}(x_{n+1})}. \quad (11)$$

This ensures detailed balance and preserves the joint invariant distribution $\pi_{\beta_0} \cdots \pi_{\beta_N}$. High-temperature replicas can explore the state space more freely, while low-temperature replicas explore local minima more thoroughly. Periodic swapping between replicas allows low-temperature simulations to escape local minima.

In *non-reversible* parallel tempering (NRPT), swaps alternate deterministically between even and odd index pairs, which improves mixing guarantees compared to reversible versions, where swap indices are chosen randomly, usually via a uniform distribution.

3.1.3 Performance Estimator

The efficiency of PT depends critically on the ability of samples to traverse the annealing path. A key performance measure is the *round trip rate*, which quantifies how often an independent sample from the reference distribution π_0 travels up to the target π_1 and back. This performance measure does not depend on the details of the local exploration move, unlike the *effective sample size* (ESS), which is commonly used to evaluate MCMC algorithms.

The round trip rate for schedule T_N is

$$\tau(T_N) = \left(2 + 2 \sum_{n=0}^{N-1} \frac{r(\beta_n, \beta_{n+1})}{1 - r(\beta_n, \beta_{n+1})} \right)^{-1}, \quad (12)$$

where

$$r(\beta, \beta') = \mathbb{E} \left[1 \wedge \frac{\pi_\beta(X') \pi_{\beta'}(X)}{\pi_\beta(X) \pi_{\beta'}(X')} \right], \quad X \sim \pi_\beta, \quad X' \sim \pi_{\beta'}. \quad (13)$$

As $N \rightarrow \infty$ and $\|T_N\| \rightarrow 0$, the asymptotic round trip rate is

$$\tau_\infty = \lim_{N \rightarrow \infty} \tau(T_N) = \frac{1}{2 + 2\Lambda}, \quad (14)$$

where $\Lambda \geq 0$ is called the *global communication barrier*.

3.1.4 Global Communication Barrier in Parallel Tempering

The *local communication barrier* is defined at annealing parameter $\beta \in [0, 1]$ as the instantaneous rejection rate of a swap between two neighboring chains:

$$\lambda(\beta) = \lim_{\delta \rightarrow 0} \frac{r(\beta, \beta + \delta)}{|\delta|}, \quad (15)$$

where $r(\beta, \beta')$ is the swap rejection probability between chains at parameters β and β' .

The *global communication barrier* is then obtained by integrating over the whole annealing path:

$$\Lambda = \int_0^1 \lambda(\beta) d\beta. \quad (16)$$

This quantity measures the cumulative communication difficulty between the reference distribution π_0 and the target distribution π , by embodying the local rejection rate across the entire path.

The authors of [?] showed that when the number of chains is finite, the rejection rate per swap can be tuned optimally:

$$N^* + 1 \approx 2\Lambda + 1, \quad (17)$$

i.e., the number of chains should be of order 2Λ , since at this scaling the average rejection rate per swap is balanced at about

$$r^* \approx \frac{\Lambda}{N}, \quad (18)$$

thus the communication is dense enough along the path to overcome the global barrier, but not wasteful in resources.

To apply parallel tempering to our problem we used the `Pigeons.jl` Julia package.

3.2 MCMCTempering.jl

Before adopting `Pigeons.jl`, we experimented with `MCMCTempering.jl` for PT. This package requires an external explorer; we used MALA with a fixed step size. It also necessitates defining a proposal distribution, leading us to implement a data-guided one:

$$q(x' | x) = \mathcal{N}\left((1 - \alpha)x + \alpha d, \sigma^2 I\right),$$

where d is the data trajectory, $\alpha \in [0, 1]$ biases toward data, and σ scales Gaussian noise. For $\alpha = 0$, it's a random walk; for $\alpha = 1$, centered on d ; intermediates balance exploration.

Advantages include manual temperature ladder control, for example $\beta_i = 0.90^i$ for $i = 0, \dots, 20$, and access to all chains via `MCMCChains.jl` for diagnostics.

Tested on a two-channel toy model, it overcame initialization in the wrong channel, converging to the data trajectory despite barriers.

However, reliance on multiple libraries (`MCMCTempering.jl`, `MCMCChains.jl`, `AdvancedMH.jl`) and lack of coherence led us to discard it for the integrated `Pigeons.jl`.

3.3 Pigeons.jl

`Pigeons.jl` is a Julia package that implements non-reversible parallel tempering (NRPT) as its primary sampler, following the theoretical recommendations of [4] (non-reversible PT with schedule optimization and monitoring of the communication barrier Λ).

Each replica is represented by a `Replica` struct that holds, among other fields, the state and the chain index. Replicas are updated *in parallel* during the exploration phase with local MCMC, and communication proposals swap adjacent chain *indices*, not full state vectors, so as to keep message sizes $\mathcal{O}(1)$ with respect to state dimension.

The *in-parallel* update of the replicas ensures that `Pigeons.jl` supports multi-threaded and MPI-based distributed execution. It is developed in such a way as to guarantee strong parallelism invariance and correctness of reductions (e.g., each replica carries its own record statistics for thread

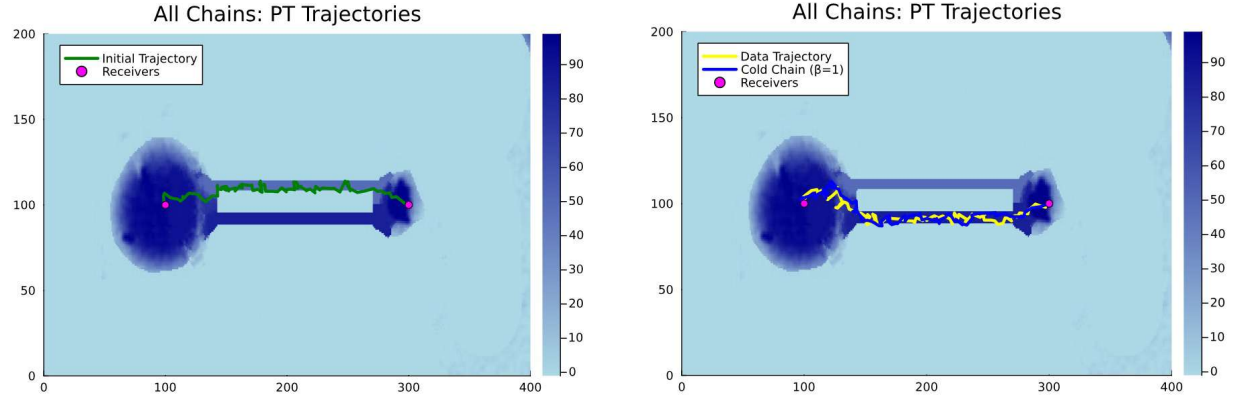


Figure 8: Toy model using `MCMCTempering.jl`: from incorrect initial (green) to accurate inferred path (blue) matching data (yellow).

safety and distribution; then they are reduced at the end of each round), in addition to $\mathcal{O}(1)$ swap message size [5].

The package exposes online quantities such as:

- an estimator for Λ ,
- the number of tempered restarts,
- swap acceptance rates (minimum / mean),
- and a normalization constant estimate.

Technical Note on `sample_iid!`

The `sample_iid!(reference_log_potential, replica, shared)` Pigeons' function is a function that is invoked only when $\beta = 0$, but it is crucial, since it is responsible for updating the replica's state by *drawing an independent sample* from the reference distribution. In fact, the purpose of tempered restarts is to allow large jumps in the state space.

Essentially, this function needs to draw a new sample v from the prior and update the replica state. However, if the reference distribution is a custom `log_potential`, then we must define our own method for `sample_iid!`. Otherwise, the default fallback behavior just performs a local MCMC step, which defeats the purpose of $\beta = 0$.

4 Description of Pigeons' Samplers

In the `Pigeons.jl` package, samplers (referred to as “explorers”) are the core Markov chain Monte Carlo (MCMC) kernels used within each chain of the parallel tempering framework. A Markov Chain is a sequence of random states (in our case, fish trajectories $x_0, x_1, \dots$) where the probability of moving to the next state depends only on the current state and not on any of the previous states, this is the Markov Property. A Markov Chain is intentionally constructed so that, after a sufficient number of steps, the probability distribution of the current state x_t no longer changes with time. This final, stable distribution is called the stationary (or invariant) distribution, so we can ensure that the samples correspond to the posterior distribution. The fundamental theorem that makes MCMC work is that the sampling algorithm (like AutoMALA or Slice Sampler) is designed with

transition rules that satisfy the Detailed Balance. These explorers generate proposals for new states in the state space, facilitating exploration of the posterior distribution. In our fish tracking project, we employ two primary explorers: AutoMALA and Slice Sampler. Each explorer is applied independently in the chains of the parallel tempering algorithm, allowing for swaps between chains to enhance mixing and overcome energy barriers in the posterior landscape. This section provides a detailed description of these samplers, starting with AutoMALA, including their mathematical foundations, operational mechanisms, and key parameters. This theoretical description is based on the paper published by Miguel Biron-Lattes et al. [2].

4.1 AutoMALA Sampler

The AutoMALA (locally adaptive Metropolis-adjusted Langevin algorithm) sampler is a gradient-based MCMC method because gradients direct the sampler to high-density regions in the target distribution. It is designed to efficiently sample from complex, high-dimensional target distributions, such as the posterior for fish trajectories in our model Eq. 1. As $\epsilon \rightarrow 0$, we perform smaller time updates to our SDE approximation, resulting in a higher acceptance probability of the proposal. This high acceptance rate comes at the expense of slow mixing due to the small step size ϵ . On the other hand, as $\epsilon \rightarrow \infty$, more distant moves are generally proposed, at the expense of a fall in the acceptance rate, leading again to slow mixing. There is therefore a tradeoff between the distance that one proposal can reach and the resulting acceptance rates. Introduced by Biron-Lattes et al. (2024), AutoMALA builds on the standard Metropolis-adjusted Langevin algorithm (MALA) but addresses a key limitation: the need for a fixed step size that may perform poorly in regions of the state space with varying scales or correlations (e.g., multimodal posteriors). Instead, AutoMALA dynamically adapts the step size at each iteration based on the local geometry of the target, while preserving the correct invariant distribution.

4.1.1 Connection between AutoMALA and HMC samplers

HMC augments the target distribution’s position variables with auxiliary momentum variables $\mathbf{p}$, treating the system as a physical one governed by Hamilton’s equations. The negative log-density of the target acts as the “potential energy” $U(\mathbf{x})$; we simulate the posterior as a potential landscape where we move according to Hamiltonian dynamics equations. The leapfrog integrator approximates the continuous dynamics discretely with a fixed step size $\epsilon > 0$. In full HMC, you chain L such leapfrog steps to simulate a trajectory of total length $L \times \epsilon$, then accept/reject the endpoint based on the change in Hamiltonian (energy). MALA (and thus AutoMALA) is equivalent to HMC with $L = 1$ (a single leapfrog step). In standard HMC we fix L but variations like NUTS double L per iteration in a recursive way to exponentially grow the explored trajectory while reusing computations from previous steps (the tree structure avoids redundant leapfrog evaluations). After n doublings, the total $L \approx 2^n$ (actually $2^n - 1$), so the computational cost (number of leapfrog evaluations, each requiring a gradient computation) is proportional to 2^n . In contrast, AutoMALA adapts the step size ϵ by successively doubling (or halving) it, but since each check uses only a single leapfrog step (as it’s MALA-based), n such adjustments cost only proportional to n . Thus, for variable targets, AutoMALA’s step-size adaptation is computationally preferable.

4.1.2 Mathematical Background

MALA is rooted in the Langevin diffusion, a stochastic differential equation (SDE) that approximates the target distribution $\pi(\mathbf{x})$. The Langevin diffusion equation is given by:

$$d\mathbf{X}_t = \frac{1}{2} \mathbf{C} \nabla \log \gamma(\mathbf{X}_t) dt + \mathbf{C}^{1/2} d\mathbf{W}_t, \quad (19)$$

where $\mathbf{W}_t$ is a Wiener process and $\mathbf{C}$ is a positive-definite preconditioning matrix (often the identity for simplicity). Under regularity conditions, this SDE has π as its stationary distribution.

The standard MALA discretizes this SDE using the Euler-Maruyama scheme with a fixed step size $h > 0$ (or equivalently $\epsilon = \sqrt{h}$) to propose a new state $\tilde{\mathbf{x}}'$ from current state $\mathbf{x}$:

$$\tilde{\mathbf{x}}' \mid \mathbf{x} \sim \mathcal{N}\left(\mathbf{x} + \frac{h}{2} \mathbf{C} \nabla \log \gamma(\mathbf{x}), h\mathbf{C}\right). \quad (20)$$

This proposal is accepted with probability $\min(1, r)$, where r is the density ratio ensuring detailed balance, or rejected (staying at $\mathbf{x}$). It is known as ULA (unadjusted Langevin Algorithm) but ULA with a fixed step h does not ensure π as stationary distribution. So MALA is a modification of it incorporating Metropolis Hasting.

AutoMALA can be seen as Hamiltonian Monte Carlo (HMC) with a single leapfrog step, augmenting the state space to $(\mathbf{x}, \mathbf{p})$ where $\mathbf{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$ and $\mathbf{M} = \mathbf{C}^{-1}$ (the “mass matrix”). The augmented target is $\pi_{\text{HMC}}(\mathbf{x}, \mathbf{p}) = \pi(\mathbf{x}) \cdot \mathcal{N}(\mathbf{p} \mid \mathbf{0}, \mathbf{M})$, allowing the use of Hamiltonian dynamics. A leapfrog integrator L_ϵ proposes $(\mathbf{x}', \mathbf{p}')$:

$$\mathbf{p}'_{1/2} = \mathbf{p} + \frac{\epsilon}{2} \nabla \log \gamma(\mathbf{x}), \quad (21)$$

$$\mathbf{x}' = \mathbf{x} + \epsilon \mathbf{M}^{-1} \mathbf{p}'_{1/2}, \quad (22)$$

$$\tilde{\mathbf{p}} = \mathbf{p}'_{1/2} + \frac{\epsilon}{2} \nabla \log \gamma(\mathbf{x}'), \quad (23)$$

$$\mathbf{p}' = -\tilde{\mathbf{p}}. \quad (24)$$

This map L_ϵ is an involution (invertible with itself as inverse) and volume-preserving, properties crucial for invariance.

The only difference between MALA and AutoMALA is the adaptance of the step size. AutoMALA further augments the state to $s = (\mathbf{x}, \mathbf{p}, a, b)$ where (a, b) are random thresholds uniformly drawn from $\Delta = \{(a, b) \in (0, 1)^2 : a < b\}$. The proposal (x', p') is then accepted with probability:

$$\alpha((x, p), (x', p')) := \min \left\{ 1, \frac{\gamma(x') \mathcal{N}(p' \mid 0, M)}{\gamma(x) \mathcal{N}(p \mid 0, M)} \right\} \quad (25)$$

In summary, autoMALA adapts ϵ “on the fly” as a function of the current state, and is carefully constructed to ensure that we still asymptotically obtain samples from the correct target distribution π .

4.1.3 How AutoMALA Works

One iteration of AutoMALA proceeds in four steps:

1. **State Augmentation:** Sample momentum $\mathbf{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$ and thresholds (a, b) uniformly from Δ . Form augmented state $s = (\mathbf{x}, \mathbf{p}, a, b)$.

2. **Step Size Selection:** Start with an initial guess $\epsilon_{\text{init}} > 0$. Compute the density ratio $r_{\text{init}} = \bar{\pi}(s')/\bar{\pi}(s)$ for proposal $s' = L_{\epsilon_{\text{init}}}(s)$. If $r_{\text{init}} \in (a, b)$, keep ϵ_{init} . If not, adjust ϵ by doubling or halving until the ratio falls appropriately relative to (a, b) :
 - If $r_{\text{init}} \geq b$ (step too small, acceptance too high), double ϵ until the ratio is strictly less than b , then halve once.
 - If $r_{\text{init}} \leq a$ (step too large, acceptance too low), halve ϵ until the ratio is strictly above a .
3. **Reversibility Check:** Compute the reverse step size $\epsilon' = \epsilon(s', \epsilon_{\text{init}})$ from proposed s' . If $\epsilon \neq \epsilon'$, reject and stay at $\mathbf{x}$. This ensures the proposal is an involution on the region where it succeeds, preserving invariance.
4. **Metropolis-Hastings Correction:** If the check passes, accept s' with probability $\alpha = \min(1, \bar{\pi}(s')/\bar{\pi}(s))$; otherwise, reject.

This adaptive process allows small steps in narrow, high-curvature regions and large steps in flat regions. Drawing the acceptance thresholds a, b uniformly generally keeps the acceptance probability bounded away from the extremes (0 and 1), as desired, without needing to specifically tune their values. For a given pair of thresholds $a < b$, if the initial acceptance ratio is too low relative to a (ϵ_{init} too large), we make the steps more conservative by successively halving ϵ until the ratio is above a (although not necessarily below b). If instead the initial acceptance ratio is too high relative to b (ϵ_{init} too small), we make the steps more aggressive by successively doubling ϵ until the ratio is below b (although not necessarily above a), and then halving the step size once for ensuring π is the stationary distribution.

4.2 Slice Sampler

This section provides a detailed explanation of how the Slice Sampler operates, including its mathematical foundations, key equations, and the parameters used in the Pigeons implementation.

4.2.1 Overview and Motivation

The Slice Sampler is a Markov chain Monte Carlo (MCMC) method designed to sample from a wide variety of distributions, particularly those where the density is known up to a proportionality constant. It serves as an alternative to methods like Gibbs sampling, which requires sampling from non-standard conditional distributions, and Metropolis-Hastings, which often needs careful tuning of proposal scales. A key advantage of slice sampling is its ability to adaptively adjust the scale of proposed changes based on the local properties of the distribution, making it more robust when the appropriate step size varies across the space.

The core idea of slice sampling is to sample uniformly from the region under the graph of the target density function. For a univariate distribution with density proportional to $f(x)$, this involves sampling points uniformly from the area $\mathcal{U} = \{(x, y) : 0 < y < f(x)\}$ and discarding the y -coordinate to obtain samples from the marginal distribution of x .

Formally, introduce an auxiliary variable y , and define the joint density over x and y as uniform over $\mathcal{U}$:

$$p(x, y) = \begin{cases} \frac{1}{Z} & \text{if } 0 < y < f(x), \\ 0 & \text{otherwise,} \end{cases}$$

where $Z = \int f(x) dx$ is the normalizing constant. The marginal density for x is then

$$p(x) = \int_0^{f(x)} \frac{1}{Z} dy = \frac{f(x)}{Z},$$

which matches the target distribution. Sampling from $p(x)$ is achieved by jointly sampling (x, y) from the uniform distribution over $\mathcal{U}$ and ignoring y .

Since direct independent sampling from $\mathcal{U}$ is often infeasible, a Markov chain is constructed that converges to this uniform distribution. This chain alternates between sampling y uniformly from $(0, f(x))$ given the current x , and updating x while leaving the uniform distribution over the “slice” $S = \{x : y < f(x)\}$ invariant.

4.2.2 Single-Variable Slice Sampling

Slice sampling is simplest for a single real-valued variable x , where the density is proportional to $f(x)$. This univariate case forms the building block for multivariate extensions, such as updating one coordinate at a time in a high-dimensional space.

Starting from the current state x_0 , a new state x_1 is generated as follows:

1. Sample y uniformly from $(0, f(x_0))$, defining the slice $S = \{x : y < f(x)\}$. Note that $x_0 \in S$.
2. Find an interval $I = (L, R)$ around x_0 that contains much or all of S .
3. Sample x_1 from the part of S within I .

For step 2, an initial interval size w (an estimate of the slice’s scale) is used. One common method is the “stepping out” procedure: Expand I in steps of size w until the endpoints are outside S . An alternative is the “doubling” procedure, which doubles the interval size until it likely encompasses S .

The doubling procedure, detailed below, expands faster than stepping out and is efficient when w underestimates the slice size:

- Randomly position an initial interval of size w containing x_0 .
- While the endpoints are inside S , double the interval by extending left or right (chosen randomly).

For step 3, sample uniformly from I and shrink I toward x_0 if the point is outside S , ensuring the process terminates. The acceptable set A is defined as

$$A = \{x : x \in S \cap I \text{ and } P(\text{Select } I \mid \text{At state } x) = P(\text{Select } I \mid \text{At state } x_0)\},$$

which maintains invariance.

This procedure leaves the target distribution invariant, as proven by detailed balance arguments in Neal (2003).

4.2.3 Multivariate Slice Sampling

In our project, the fish trajectory is multivariate ($x = (x_1, \dots, x_n)$ representing positions over time), so we extend slice sampling to higher dimensions. One approach is to apply single-variable updates to each coordinate sequentially, akin to Gibbs sampling but with adaptive steps.

A direct multivariate method samples from the slice $S = \{x : y < f(x)\}$ using a hyperrectangle $H = \prod_{i=1}^n (L_i, R_i)$ around the current point x_0 :

1. Sample y uniformly from $(0, f(x_0))$.
2. Find H using scale estimates w_i for each dimension, randomly positioning H to contain x_0 . Expansion via doubling can be used, stopping when a random point in H is outside S .
3. Sample from the part of S within H , shrinking H if needed. To adapt to dependencies, shrink along the axis maximizing $(R_i - L_i)|\nabla_i \log f(x)|$, where ∇_i is the partial derivative, estimating the direction of greatest density change.

This multivariate approach can respect correlations between variables, potentially allowing larger moves than coordinate-wise updates. More advanced variants use local quadratic approximations for better adaptation, though these require gradient evaluations.

5 Explorers comparison

5.1 AutoMALA in Pigeons

Although its default behaviour usually avoids the need for manual tuning, several optional parameters can be specified. The most relevant ones for our study are:

- **base_n_refresh**: Base number of steps (or momentum refreshments) between swaps. This value is later scaled with dimensionality through **exponent_n_refresh**.
- **exponent_n_refresh**: Exponent that controls how the number of refreshes scales with the problem dimensionality.
- **default_autodiff_backend**: Backend to be used for automatic differentiation. It can be ignored when analytical gradients are provided.
- **step_size**: Initial step size for the automatic adjustment procedure. It is updated dynamically during the simulation.
- **preconditioner**: Strategy for defining a preconditioning matrix. When not specified, the identity is used initially.
- **estimated_target_std_deviations**: Updated after the first iteration to improve preconditioning. Defaults to an identity-based mass matrix before empirical estimates are available.

5.1.1 Grid Search for Parameter Tuning

We carried out several grid searches; here we present the final one that yielded the most stable and efficient configuration. Previous exploratory runs are omitted for clarity.

Table 2 shows the results of the parameter combinations tested. The performance is reported in terms of elapsed runtime and the cold chain log-posterior value (`logpost_cold`). Initial trajectory log-posterior value (`logpost_init`) and data trajectory log-posterior value (`logpost_depth`) are obviously fixed among the grid search:

$$\text{logpost_init} = -646.249 \quad \text{logpost_depth} = -464.385$$

The results reveal that:

step_size	base_n_refresh	exponent_n_refresh	elapsed_time	logpost_cold
2.5	10	0.5	870.258	-455.373
2.5	15	0.5	1180.916	-465.643
2.5	30	0.5	1954.510	-474.522
5.0	10	0.5	852.764	-455.373
5.0	15	0.5	1095.552	-465.643
5.0	30	0.5	1932.638	-474.522

Table 2: Final grid search results for AutoMALA parameters. Higher logpost_cold indicates better sampling quality.

- Smaller values of `base_n_refresh` consistently led to better efficiency (shorter runtime and higher log-posterior values).
- Increasing `base_n_refresh` beyond 10 only added computational cost without improving the quality of the trajectory inference.
- Both tested `step_size` values (2.5 and 5.0) performed comparably, the explanation resides in the definition of AutoMALA, the value of `step_size` provided to AutoMALA sampler is used only at the first iteration and is updated automatically for subsequent iterations.
- The exponent for dimensionality scaling was kept fixed at 0.5, as varying it did not significantly impact results in our setup.

The best-performing configuration was obtained with:

`step_size = 2.5, base_n_refresh = 10, exponent_n_refresh = 0.5`

achieving a cold-chain log-posterior value of -455.37 , which represents the most accurate and efficient trajectory inference within our tested grid.

5.2 Slice Sampler in Pigeons

The Pigeons package implements Neal’s slice sampler with the following parameters:

- `w`: Initial slice size, estimating the typical scale of changes.
- `p`: Maximum expansion factor; slices are no larger than $2^p \cdot w$.
- `n_passes`: Number of passes through all variables per exploration step, controlling how many coordinates are updated per iteration.
- `max_iter`: Maximum iterations in the shrinkage procedure before erroring out, preventing infinite loops.

In our PT setup, the slice sampler acts as an explorer within each chain, proposing moves that adapt to the local posterior landscape. This helps overcome energy barriers through chain swaps, leading to a refined trajectory from the cold chain.

5.2.1 Grid Search for Parameter Tuning.

We carried out several grid searches; here we present the final one that yielded the most stable and efficient configuration. Previous exploratory runs are omitted for clarity.

The initial trajectory log-posterior values are fixed across all runs:

$$\text{logpost_init} = -695.650 \quad \text{logpost_depth} = -482.645$$

Therefore, we only report elapsed runtime and the cold chain log-posterior (`logpost_cold`).

Table 3 shows the tested parameter combinations for `n_passes` and `max_iter`. The performance is reported in terms of runtime and `logpost_cold`. After identifying the best configuration for these two parameters, we fixed them and performed a second grid search for `w` and `p`. Since the internal adaptation of these values provided consistently better performance in our experiments, we do not report explicit values for them and instead rely on the sampler’s internal adjustment mechanism gave better results for all the tests.

<code>n_passes</code>	<code>max_iter</code>	<code>elapsed_time</code>	<code>logpost_cold</code>
1	50	187.997	-476.353
1	100	183.959	-476.353
2	50	204.087	-468.572
2	100	188.695	-468.572
10	50	404.690	-460.097
10	100	457.727	-460.097

Table 3: Final grid search results for SliceSampler parameters. Higher `logpost_cold` indicates better sampling quality.

The results reveal that:

- Increasing `n_passes` significantly improves the log-posterior value of the inferred trajectory, with the best performance at `n_passes=10`.
- The parameter `max_iter` had negligible influence on the quality of the results, although larger values increased computational cost.
- The cold chain log-posterior improves consistently as `n_passes` increases, suggesting that more thorough coordinate updates are beneficial for our inference problem.
- In the subsequent grid search, leaving `w` and `p` adaptive to the problem gave better performance than fixing explicit values.

The best-performing configuration was obtained with:

$$\text{n_passes} = 10, \quad \text{max_iter} = 50$$

achieving a cold-chain log-posterior value of -460.10 , which represents the most accurate trajectory inference within our tested grid.

5.3 AutoMALA vs Slice Sampler

To assess the relative performance of the two explorers, we challenged them under identical conditions: the same initial trajectory, the same data trajectory, and identical Parallel Tempering

settings (`nchains = 6`, `nrounds = 11`). This setup allows for a fair evaluation of which sampler yields the most accurate inference for the fish trajectory reconstruction problem.

The best configurations, as determined from the previous grid searches, were:

AutoMALA: `step_size = 2.5`, `base_n_refresh = 10`, `exponent_n_refresh = 0.5`

SliceSampler: `n_passes = 10`, `max_iter = 50` (`w` and `p` adjusted internally)

The resulting log-likelihood values are shown in Table 4. For reference, we also include the log-likelihood of the initial and data trajectories.

Sampler	Log-likelihood
AutoMALA (cold chain)	-452.430
SliceSampler (cold chain)	-474.805
Initial trajectory	-758.478
Data trajectory	-525.870

Table 4: Log-likelihood comparison between AutoMALA and SliceSampler. Higher values indicate better trajectory inference.

Figure 9 illustrates the spatial reconstruction at 200m depth in the Firth of Lorn. The initial trajectory is shown in red, the data trajectory in green, and the final cold-chain trajectories inferred by Parallel Tempering using AutoMALA in yellow and SliceSampler in blue.

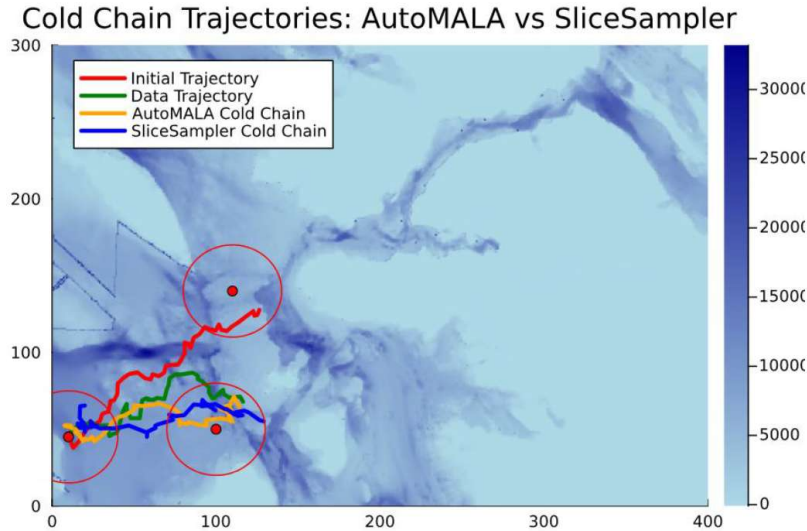


Figure 9: Map of Firth of Lorn at 200m depth showing the initial trajectory (red), the data trajectory (green), and the cold-chain reconstructions from AutoMALA (yellow) and SliceSampler (blue).

The comparison highlights that AutoMALA achieves the best log-likelihood (-452.43), outperforming SliceSampler (-474.80). Both samplers successfully optimize the initial trajectory towards the data trajectory, but AutoMALA yields a more accurate posterior estimate. This advantage comes at the cost of runtime: AutoMALA required 870.258 seconds, while SliceSampler converged in 404.690 seconds.

The difference stems from the fact that AutoMALA is a gradient-based MCMC method. By exploiting local gradient information, it is guided more efficiently towards high-density regions of the posterior, resulting in superior sampling quality. The use of `auto_enzyme` for gradient computation significantly reduces computational overhead compared to finite-difference methods such as `forward_diff`, but AutoMALA still requires more time than SliceSampler.

In practice, both samplers perform robustly. SliceSampler provides faster exploration and may be more suitable when sampling very large numbers of trajectories, while AutoMALA should be preferred when higher accuracy in the inferred trajectory is required. Ultimately, the choice of explorer is task-dependent, balancing accuracy against computational cost.

6 Sampling results

6.1 Toy bathymetry toy data

We start the discussion by proving the inefficiency of a standard MCMC sampler in solving the addressed problem. `Pigeons.jl`, in fact, when setting `n_chains=1`, just runs a standard single-chain MCMC sampler, using the specified kernel.

We do so in two scenarios:

- Starting from the bathymetry shown in Figure 1, we perform 100 single-chain runs for 10 rounds (which means 1025 scans) using as initial trajectory a path that, partly, goes through a different channel than the “real” trajectory Figure 10. The result Figure 10 shows the stiffness of the sampler. A more effective way to perceive this is to analyze the depth values associated with the reconstructed trajectories Figure 11.
- We study a much simpler configuration: we consider a bathymetry profile with just 2 channels of different depth, choosing the initial and real trajectories so that they are in different channels Figure 12. We confirm what was previously obtained Figure 12, in fact not only the trajectory is not able to reach the right channel, but also the logposterior evolution manifests bad sampling, since it shows significant correlation in the evolution.

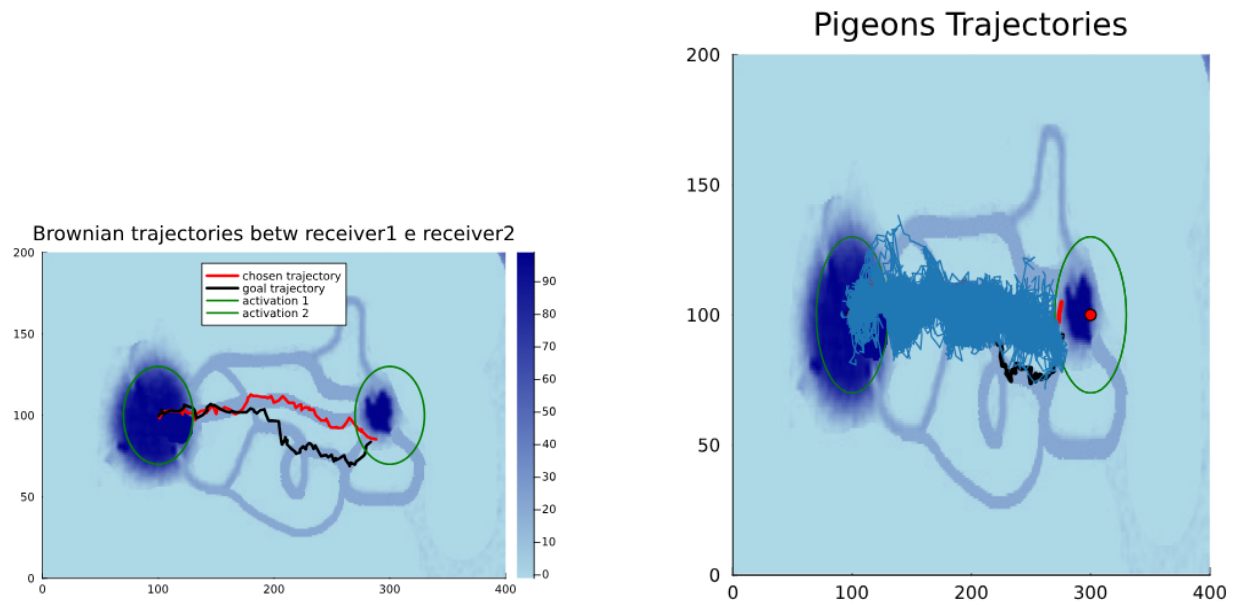


Figure 10: On the left the starting and the real trajectories, on the right the 100 paths reconstructed by a standard MCMC sampling from that initial trajectory. These show the stiffness of a standard MCMC method applied in a scenario as the present one.

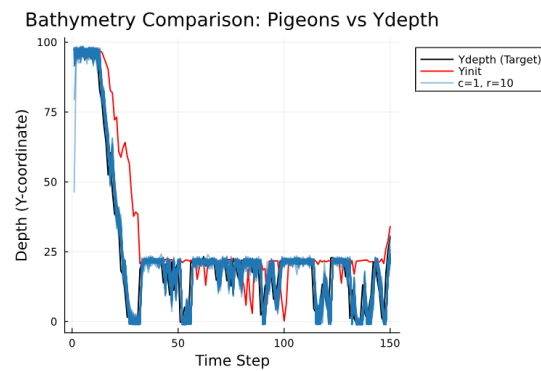


Figure 11: Bathymetry profile of the reconstructed trajectories

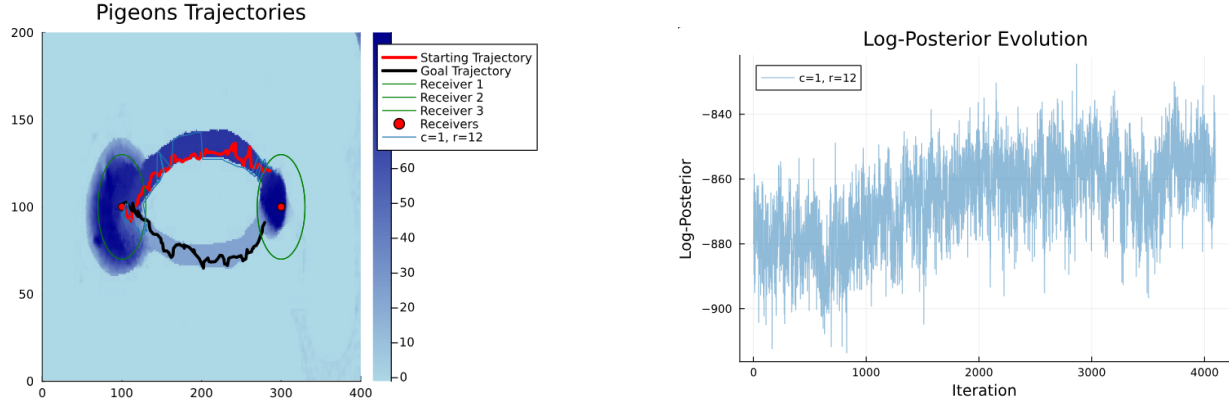


Figure 12: Left: trajectory reconstruction by a standard single chain MCMC sampler; "c" stands for n-chains, "r" for n-rounds. Right: logposterior evolution of the last sampling iteration over the cold chain.

We then apply PT. To determine the number of chains to use, we perform a coarse-grained grid search on `n_chains`. First we note that even with fewer rounds compared to the standard MCMC (9, so 512 scans), all the reconstructed trajectories end up in the right channel (Figure 13). Regarding Λ , we observe that its value stabilizes around 9–10, but we also see that using a high number of chains beyond a certain threshold does not lead to an increase in performance (Figure 15), and indeed, increases the computation time. It is therefore considered appropriate to use the minimum number of chains such that it is approximately equal to 2Λ , as prescribed by [4]. To determine this value more precisely, we perform a finer grid search on the values of `n_chains`, and we find that the optimal value to use in this scenario is around 18 chains (Table 5, Figure 14).

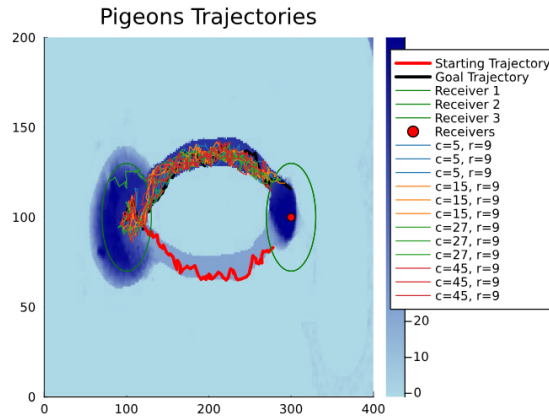


Figure 13: Trajectory reconstruction via a coarse grained grid search on the number of chains. Each configuration of chains and rounds appears 3 times because each run is repeated 3 times.

scans	$\wedge$				
	c=8	c=10	c=14	c=18	c=20
2	3	3	5	7	11
4	1.5	1	2.86	3.96	3
8	3.04	2.5	2.98	2.2	2.43
16	3.79	3.68	4.65	2.73	2.59
32	5.56	5.78	5.77	7.08	5.32
64	6.06	6.43	7.55	8.48	7.40
128	6.20	6.79	8.24	9.30	8.52
256	6.19	6.90	8.11	9.53	8.98
512	6.30	7.08	8.31	9.82	9.03

Table 5: Global communication barrier estimations for varying number of chains in the 2-channels toy bathymetry using toy data.

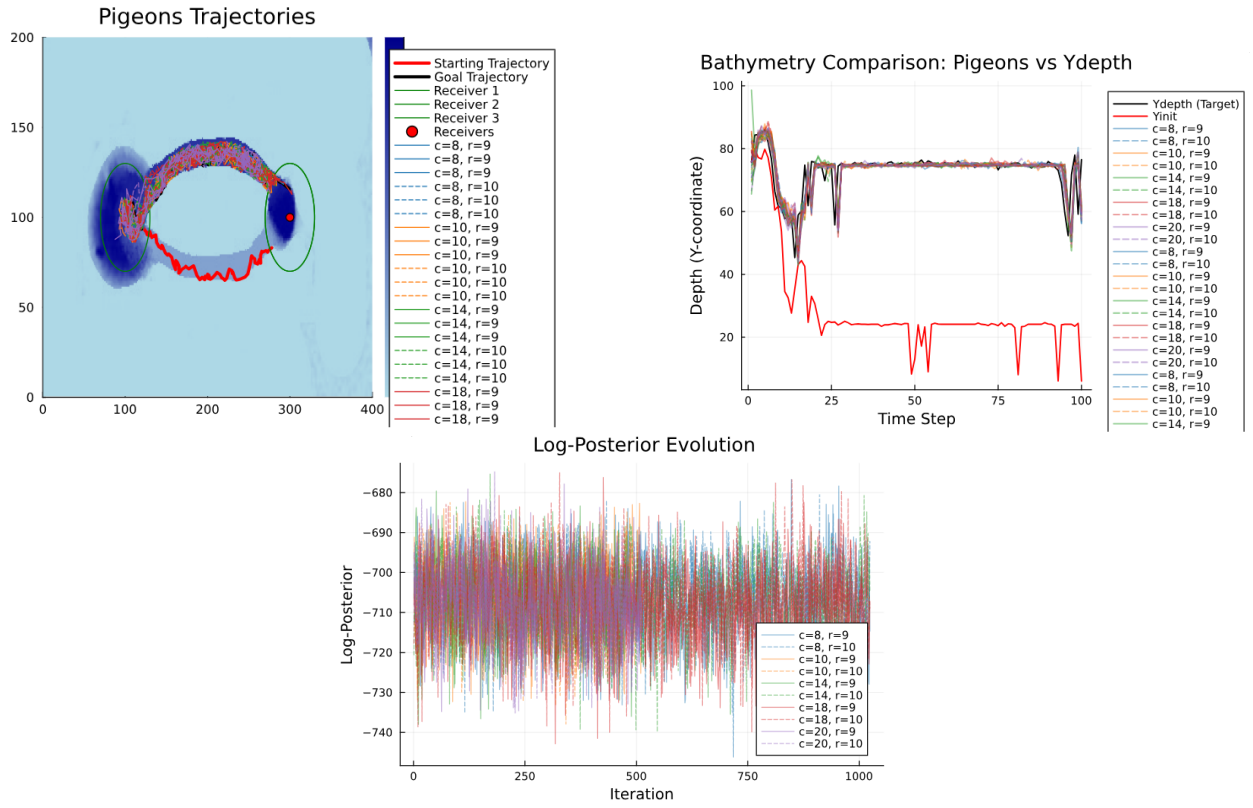


Figure 14: From the left to the right: trajectory reconstruction via a fine grained grid search on the number of chains, depth data associated to the reconstructed trajectories, logpost sampling evolution of the last iteration of the "cold-chain". Knowing that the single traces are not distinguishable, these plots are mostly meant to show the overlap between the various traces.

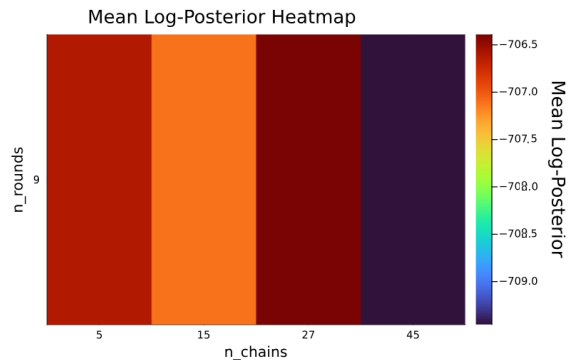


Figure 15: Heatmap plot that shows the log-posterior value, as a function of (n-chains, n-rounds), averaged over all iterations of the cold chain. Note the small range of the colorbar.

6.2 Real Bathymetry & Toy Data

Here we apply PT directly, and after an analogous grid search on `n_chains`, we get that the best configuration is `n_chains=6`, which produces the result shown in Figure 16. As we can see from the figure, even in this case the initial trajectory is on purpose very different from the goal one, and yet the algorithm has been able to correctly solve the problem.

Technical note on `sample_iid!`. It is important to note that in both TT and RT we decided to approximate sampling from the reference distribution by randomly choosing one of the thousands of initial trajectories generated. This is because the reference distribution we are using represents an unbiased random walk, where, however, the initial trajectories are biased. Indeed, it has been observed that letting `sample_iid!` sample from this reference distribution makes the algorithm unstable: the number of restarts always remains at 0, and λ varies greatly between runs as the chains vary. This approach instead grants stable performance in terms of λ and a consistent number of tempered restarts Figure 17.

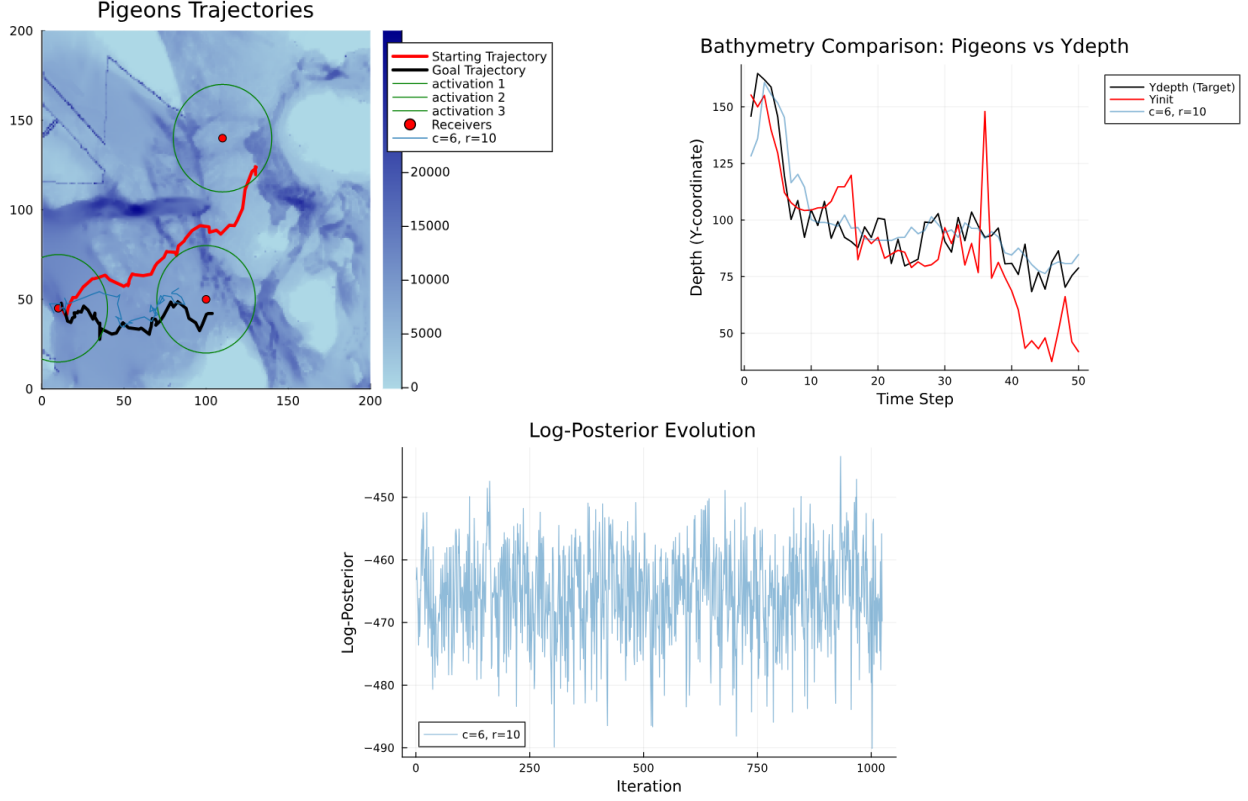


Figure 16: Analogous plots to those of Figure 14, but for the real bathymetry

scans	restarts	Λ	time(s)	$\log(Z_1/Z_0)$
2	0	1.38	0.453	-235
4	0	1.5	1.76	-231
8	0	2.87	0.596	-224
16	1	1.98	0.758	-214
32	1	2.14	0.504	-209
64	1	2.29	0.913	-209
128	9	2.16	1.93	-210
256	13	2.31	3.21	-211
512	36	2.07	6.18	-209
1.02e+03	64	2.27	11.9	-210

Figure 17: Run recap plotted by Pigeons.jl. The last column represent the normalization constants estimations' log ratio. This table is particularly meant to show the significant number of tempered restarts.

6.3 Real Bathymetry & Real Data

In this section, we extend our analysis by applying the results from the previous chapters to real data. In this scenario, we no longer have the assurance that there exists a path, or a family of paths, that satisfies both the depth and acoustic data, as the fish can swim freely. For instance, we might observe the activation of a receiver in a very deep area while the depth data indicate near-surface positions. First, we implemented a function that interprets the bathymetry data, which is discretized into pixels, and interpolates it to create a continuous map. Such a map can be zoomed

freely, with plots adjusting accordingly. Next, we need to estimate an initial trajectory, similar to what was done for the bridges function in the toy model. The key difference is that we now have a list of 28 receivers instead of 2, and we must ensure that the fish moves sequentially from one to another without sudden jumps, which would render the trajectory less plausible.

Here, we discuss the results obtained using real data. First, a series of runs was conducted to determine the optimal number of chains, and we found that the sweet spot occurs at 9 chains. For computational reasons, we limited this analysis to 100 data points, corresponding to 100 pairs of (x, y) coordinates. These were obtained by chunking the trajectory starting from point 4.3. We are free to choose both the length and the initial point; accordingly, we decided to conduct the analysis with initial points occurring during a receiver activation.

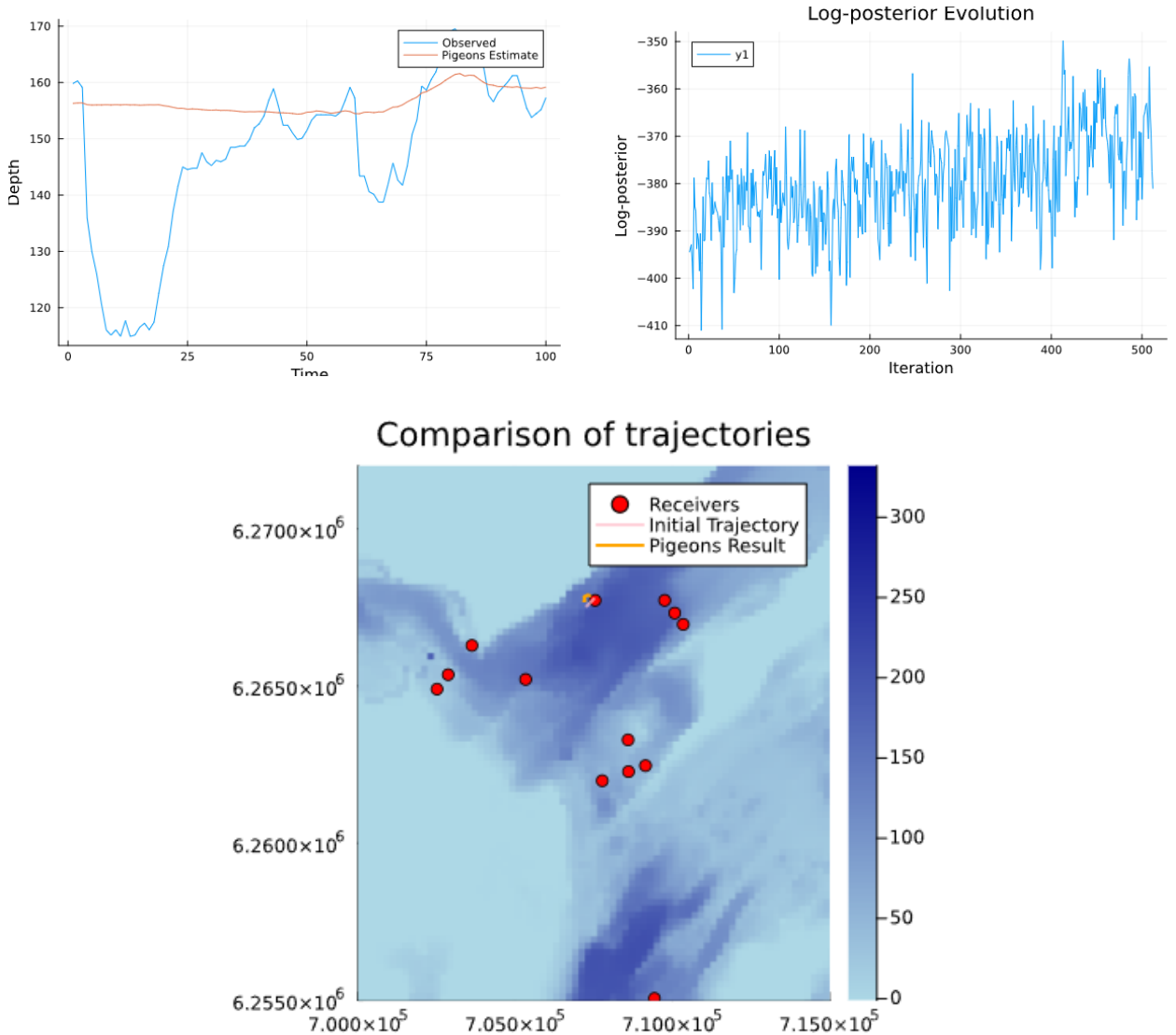


Figure 18: Trajectory reconstruction, depth profile, and log-posterior for a 100-data-point subset starting at a receiver activation. Note that the depth reconstruction does not perfectly match the data.

Note that the depth reconstruction does not accurately match the data. However, as discussed above, this may not be an issue provided that the discrepancy arises from the acoustic data con-

straining the path to remain in the appropriate region. To test this hypothesis, we performed the same run but manipulated the data to include only non-active receivers. The results are as follows, as we can see the fish stays around bay area to better satisfy the depth profile:

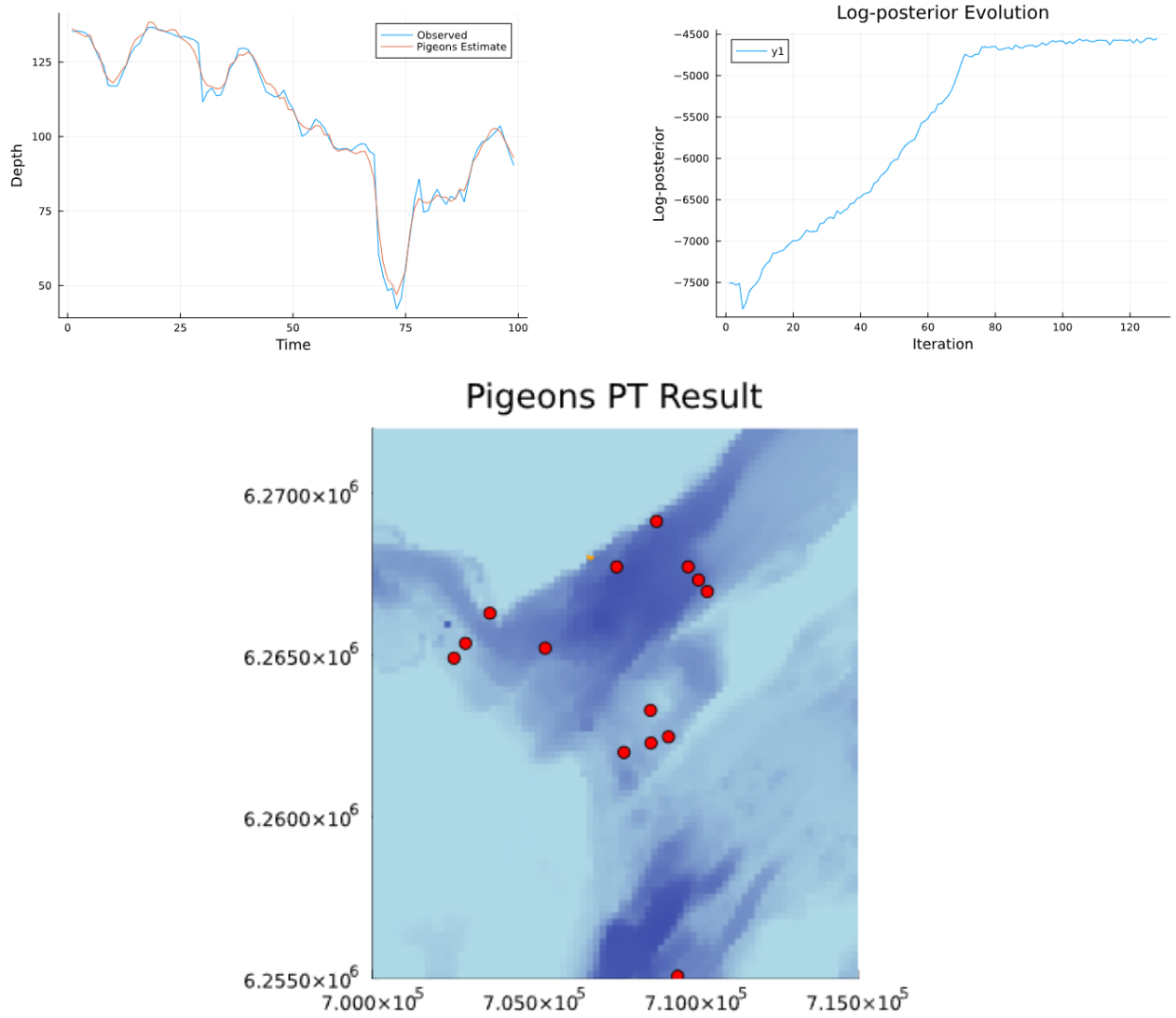


Figure 19: Results for a 100-data-point run using manipulated data (non-active receivers only). The trajectory correctly remains near the bay area to better satisfy the depth profile constraints.

With this in mind we proceeded to make a longer run, composed by 500 data points.

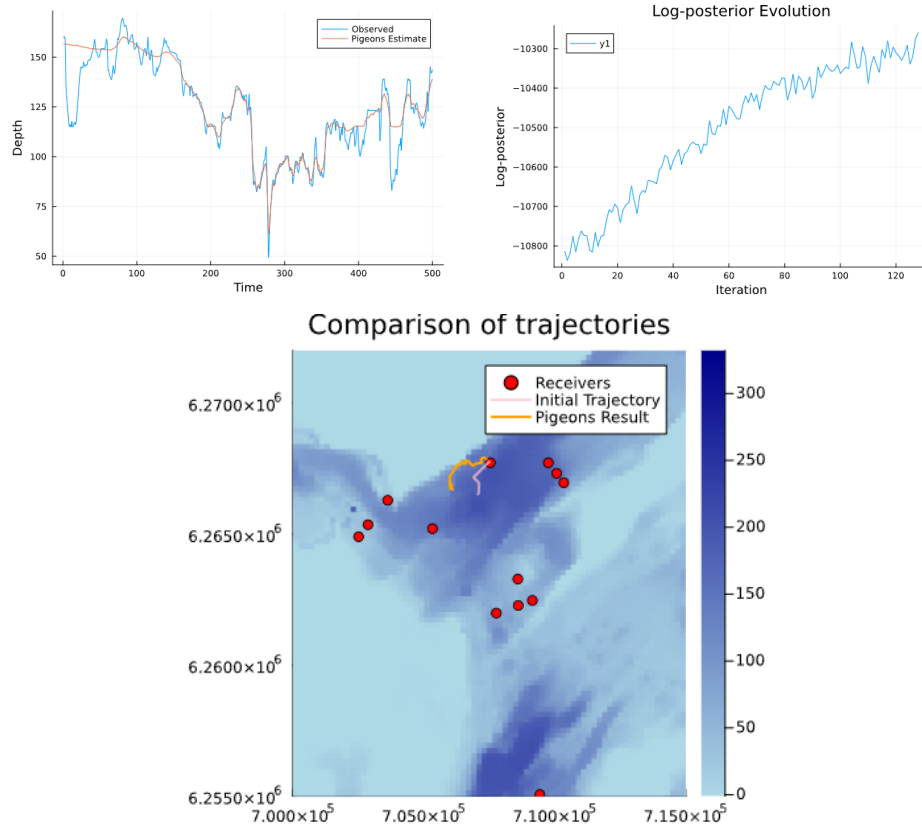


Figure 20: Trajectory reconstruction, depth profile, and log-posterior for an extended run utilizing 500 data points.

6.3.1 Persisting Issues

The mixing of chains is still not optimal, as we were unable to obtain any restarts in the runs. This means that we are not truly utilizing parallel tempering, since the target is not connected to the reference, and thus it is not influencing the evolution of the cold chain.

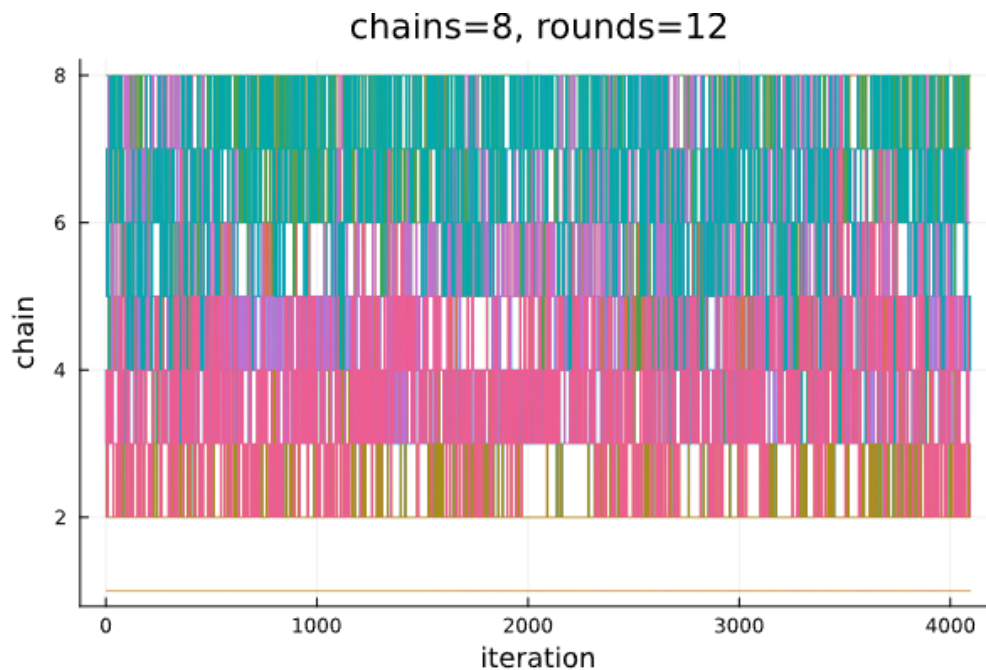


Figure 21: Diagnostics illustrating suboptimal chain mixing. The absence of tempered restarts indicates that the cold chain is not being fully influenced by the reference distribution.

References

- [1] Uffe Høgsbro Thygesen, Martin Wæver Pedersen, and Henrik Madsen. Geolocating fish using hidden Markov models and data storage tags. In J.L. Nielsen et al. (eds.), *Tagging and Tracking of Marine Animals with Electronic Devices*, pages 277–293. Springer, 2009.
- [2] Miguel Biron-Lattes, Nikola Surjanovic, Saifuddin Syed, Trevor Campbell, and Alexandre Bouchard-Côté. autoMALA: Locally adaptive Metropolis-adjusted Langevin algorithm. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages TBD. PMLR, 2024.
- [3] Saifuddin Syed, Vittorio Romaniello, Trevor Campbell, and Alexandre Bouchard-Côté. Parallel Tempering on Optimized Paths. arXiv preprint arXiv:2102.07720, 2021. <https://arxiv.org/abs/2102.07720>.
- [4] Saifuddin Syed, Alexandre Bouchard-Côté, George Deligiannidis, and Arnaud Doucet. Non-Reversible Parallel Tempering: A Scalable Highly Parallel MCMC Scheme. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 84(2):321–350, 2021. <http://dx.doi.org/10.1111/rssb.12464>.
- [5] Nikola Surjanovic, Miguel Biron-Lattes, Paul Tiede, Saifuddin Syed, Trevor Campbell, and Alexandre Bouchard-Côté. Pigeons.jl: Distributed Sampling From Intractable Distributions. arXiv preprint arXiv:2308.09769, 2023. <https://arxiv.org/abs/2308.09769>.
- [6] Andreas Scheidegger. Starfish Julia Package Github Repository. <https://github.com/scheidan/Starfish.jl/tree/main>.